读懂 MoE 之前，你需要知道的事

零基础入门指南 · 从神经网络到线性注意力，22 个概念，一步一步讲清楚

目录

1. 神经网络是什么
2. 权重（Weight）是什么
3. 训练是怎么回事
4. Token 是什么
5. 向量（Vector）是什么
6. 矩阵乘法是什么
7. Transformer 是什么
8. Attention 是什么
9. FFN（全连接网络）是什么
10. 参数量是什么
11. 稠密模型 vs 稀疏模型
12. MoE 是什么
13. 训练 vs 推理
14. 层次化路由是什么
15. 专家容量是什么
16. 细粒度专家切分是什么
17. 共享专家是什么
18. 无辅助损失的负载均衡是什么
19. MLA（多头潜注意力）是什么
20. KV Cache 是什么
21. RLHF / PPO / GRPO 是什么
22. 线性注意力是什么

第 1 章　神经网络是什么

它是什么

神经网络是一种让计算机"学会做事"的方法。你不需要告诉它每一步怎么做，只需要给它大量例子，它自己从例子里总结规律。

> 💡 **类比：** 你教小孩认猫，不会给他一本"猫的定义手册"，而是指着猫说"这是猫"，指着狗说"这不是猫"。看多了，小孩自然就学会了。神经网络也是这样——靠大量例子学习，而不是靠规则。

神经网络由很多层组成，每层做一点点变换，层层叠加，最终得出结果：

层	作用	类比
输入层	接收原始数据	耳朵/眼睛，负责接收信息
隐藏层	提取特征、变换信息	大脑，负责理解和加工
输出层	给出最终结果	嘴巴，负责说出答案

为什么叫"神经"网络

因为它的设计灵感来自人脑的神经元。人脑里有约 860 亿个神经元，每个神经元接收来自其他神经元的信号，处理后再传给下一个。神经网络模仿了这个结构——每个"节点"接收输入、做计算、传给下一层。

没有神经网络会怎样

以前的 AI 靠人工写规则，比如"如果句子里有'猫'这个字，就判断为动物类"。规则写不完，也写不准。神经网络让计算机自己从数据里学规则，彻底改变了 AI 的能力上限。

> ⚡ **核心一句话：** 神经网络 = 很多层的数学变换，每层把输入变成新的表示，最终得出答案。

第 2 章　权重（Weight）是什么

它是什么

权重就是神经网络里的一堆数字。每两个节点之间的连接都有一个权重，表示"这个连接有多重要"。

> 💡 **类比：** 你雇了一个翻译，第一天什么都不会，乱翻。客户说"这里翻错了"，翻译记下来，下次改一点。反复纠正几万次之后，翻译越来越准。翻译脑子里积累的经验，就是权重。

权重是模型"存知识"的地方。一个 7B 的模型，就是有 **70 亿个**这样的数字。这 70 亿个数字里，存着模型学到的所有知识——语法、常识、代码、数学……

没有权重会怎样

没有权重，神经网络就是一堆空架子，什么都不知道。训练的本质，就是把这些数字从随机值调整成"有意义的值"。

> ⚡ **核心一句话：** 权重 = 模型的知识。训练 = 把权重从随机数调成有用的数。

第 3 章　训练是怎么回事

训练的完整流程

训练就是让模型反复"做题、对答案、改错"的过程，分四步：

步骤	做什么	类比
① 喂数据	把大量文本输入模型	给学生发试卷
② 算 Loss	对比模型输出和正确答案，算误差	批改试卷，算错了几分
③ 反向传播	从结果倒推哪些权重导致了误差	找出哪道题做错了、为什么错
④ 更新权重	用优化器调整权重，减小误差	学生改错，下次做对

重复以上四步，几万亿次。

Loss 是什么

Loss（损失）就是误差有多大。模型输出的答案和正确答案之间的差距，用一个数字表示。**Loss 越小，模型越准。** 训练的目标就是让 Loss 不断变小。

> 💡 **类比：** 考试得了 60 分，Loss = 40（满分 100）。训练就是让这个分数越来越高，Loss 越来越小。

反向传播是什么

反向传播是一种数学方法，用来找出"哪些权重对 Loss 的贡献最大"。知道了哪里出问题，才能有针对性地调整。

> 💡 **类比：** 你做了一道菜，客人说太咸了。反向传播就是帮你找出"是加盐那一步出了问题"，而不是把整个菜谱都改一遍。

梯度是什么

梯度是反向传播算出来的结果，告诉你每个权重"应该往哪个方向调、调多少"。梯度大 = 这个权重影响很大，需要大幅调整；梯度小 = 影响不大，小幅调整。

优化器是什么

优化器是执行权重更新的工具。常见的有 **SGD**（随机梯度下降）和 **Adam**。梯度告诉它往哪改，优化器决定改多少、怎么改。

> ⚡ **核心一句话：** 训练 = 喂数据 → 算 Loss → 反向传播算梯度 → 优化器更新权重。重复几万亿次。

第 4 章　Token 是什么

它是什么

Token 是模型"读"文本的最小单位。模型不直接处理汉字或字母，而是先把文本切成一个个 token。一个 token 大约等于 0.75 个英文单词或 1–2 个汉字。

> 💡 **类比：** 你看书不是一字一字看，也不是一页一页看，而是一个"词块"一个词块地理解。Token 就是模型的"词块"。

原文	Token 化后
`MoE is great`	`["Mo", "E", " is", " great"]`
混合专家模型	`["混合", "专家", "模型"]`

为什么 MoE 里 token 很重要

MoE 的 router（路由器）**以 token 为单位做决策**：每一个 token 进来，router 决定把它发给哪几个 expert 处理。不同的 token 可能走完全不同的 expert——"猫"走 expert #3，"微积分"走 expert #7。

> ⚡ **核心一句话：** Token = 模型处理的最小单元。MoE 的路由决策是 token 级别的。

第 5 章　向量（Vector）是什么

它是什么

向量就是一组有序的数字。模型里所有的东西——token、词、句子——都被表示成向量。

> 💡 **类比：** 你要在淘宝搜一双鞋，不能把鞋的照片直接塞进搜索框。你只能描述"颜色=黑、尺码=42、风格=运动"。这个描述就是这双鞋的"向量"——用一组特征来表示一个东西。

embedding 是什么

把 token 转换成向量的过程叫 embedding。转换后的向量叫 embedding vector。语义相近的词，向量也相近——"猫"和"狗"的向量距离很近，"猫"和"微积分"的向量距离很远。

为什么 MoE 需要向量

Router 通过计算 token 向量和每个 expert 的"专长向量"之间的相似度，来决定把 token 发给谁。整个过程就是向量运算。

> ⚡ **核心一句话：** 向量 = 用数字表示万物。MoE 的 router 靠向量运算选 expert。

第 6 章　矩阵乘法是什么

它是什么

矩阵是一堆数字排成表格。矩阵乘法就是把两个表格按规则相乘，得到一个新表格。神经网络的每一层，本质上就是一次矩阵乘法。

> 💡 **类比：** Excel 里，你把"单价"列和"数量"列相乘得到"总价"列。矩阵乘法就是这种批量运算的升级版——几百行、几百列同时算。

为什么是"一切的基础"

一个 Transformer 层 99% 的计算是矩阵乘法。GPU 就是专门为矩阵乘法设计的——它有成百上千个核心，同时做乘法，所以在矩阵乘法上极快。

对 MoE 的意义

MoE 的稀疏性本质上就是"只做一部分矩阵乘法"——不是把所有 expert 的矩阵都算一遍，只算被选中的那几个。这就是 MoE 省计算量的根本原因。

> ⚡ **核心一句话：** 神经网络 = 矩阵乘法流水线。GPU 就是矩阵乘法加速器。

第 7 章　Transformer 是什么

它是什么

Transformer 是 2017 年 Google 提出的神经网络架构，几乎所有大语言模型（GPT、Claude、Gemini、DeepSeek）都基于它。它由两层交替堆叠而成：

```
Token 向量
  → Attention 层（理解上下文关系）
  → FFN 层（存储知识、做变换）
  → Attention 层
  → FFN 层
  → ...（重复 N 次）
  → 输出
```

一个 7B 模型大约有 32 层（即 32 个 Attention + 32 个 FFN）。

MoE 替换了什么

MoE 只替换 Transformer 中的 FFN 层。 Attention 层保持不变。原来的一个 FFN 被替换成"一个 router + 多个 expert（每个 expert 就是一个 FFN）"。

```
原始 Transformer:          MoE Transformer:
  Attention → FFN            Attention → Router → Expert #3, #7 → 合并
```

> ⚡ **核心一句话：** Transformer = Attention + FFN 交替堆叠。MoE 只替换 FFN，不碰 Attention。

第 8 章　Attention 是什么

它是什么

Attention（注意力）让模型"看上下文"。读到"她把苹果吃了"，Attention 能让模型理解"她"指谁、"苹果"是吃的不是手机。

> 💡 **类比：** 你读一篇文章时，不会孤立地看每个词，而是根据上下文理解。Attention 就是模型的"上下文理解能力"。

Self-Attention 怎么工作

每个 token 生成三个向量：Query（我在找什么）、Key（我有什么）、Value（我的内容）。Q 和 K 做点积算出"相关性分数"，分数越高表示两个 token 关系越紧密，然后用分数加权 V。

MHA / MQA / GQA

变体	含义	谁在用
MHA	多组 Q/K/V 并行，每组关注不同方面	原始 Transformer、GPT-3
MQA	所有 head 共享一组 K/V，省显存	PaLM
GQA	几个 head 共享一组 K/V，折中方案	LLaMA、DeepSeek

为什么 MoE 关心 Attention

Attention 消耗了大量 KV Cache 显存。MoE 虽然减少了 FFN 计算量，但 **Attention 的 KV Cache 开销不变**。这是 MoE 推理时显存压力的主要来源之一。

> ⚡ **核心一句话：** Attention 让模型理解上下文。MoE 不替换 Attention，两者配合：Attention 负责"理解"，MoE-FFN 负责"存储和计算"。

第 9 章　FFN（全连接网络）是什么

它是什么

FFN（Feed-Forward Network）就是最简单的神经网络：输入 → 放大 → 激活函数 → 缩回 → 输出。一个 Transformer 层里，Attention 先干活，然后 FFN 接着处理。

> 💡 **类比：** Attention 是"查阅上下文"，把相关信息收集起来。FFN 是"专家大脑"，基于收集到的信息做判断、存储知识。

为什么 FFN 存知识

研究发现，Transformer 里的事实性知识（"巴黎是法国首都"这类）主要存在 FFN 的权重里。Attention 负责检索，FFN 负责回忆。

FFN 和 MoE 的关系

MoE 的每个 expert **就是一个独立的 FFN**。Router 决定每个 token 走哪几个 FFN expert：

```
MoE 的 FFN = Router + [FFN_1, FFN_2, FFN_3, ..., FFN_N]
                          ↓
                  每个都是一个"专家"
```

想让 expert #5 擅长数学？训练时多喂数学数据，router 自己会学会把数学相关的 token 发给它。

> ⚡ **核心一句话：** MoE 的 expert = 一个 FFN。N 个 expert = N 个小专家，各有所长。

第 10 章　参数量是什么

它是什么

参数量 = 模型所有权重的总数，单位是 B（十亿）。7B = 70 亿个数字。

参数量的意义	说明
决定了模型的"脑容量"	参数越多，能记住的知识越多
决定了显存需求	1B 参数 ≈ 2GB 显存（FP16）
决定了推理速度	参数越多，算得越慢

稠密模型 vs MoE 的参数量

	稠密模型（如 LLaMA-7B）	MoE 模型（如 Mixtral 8x7B）
总参数	7B	47B（8 × 6B + router）
每次激活	7B	13B（2 个 expert + shared）
显存需求	~14 GB	~94 GB
推理速度	参考基准	近似于 13B 稠密模型

核心矛盾： MoE 用更大的总参数获得了更强的能力，但推理时必须把所有 expert 加载到显存。参数大了 6 倍，显存也大了 6 倍。

> ⚡ **核心一句话：** MoE 总参数量大、激活参数量小。训练和推理的关键差异就在这里。

第 11 章　稠密模型 vs 稀疏模型

稠密模型

每个 token 都会激活模型的**所有参数**。一个 7B 的稠密模型，处理任何一个 token 时，70 亿个权重全部参与计算。

稀疏模型（MoE）

每个 token 只激活**一部分参数**（被选中的几个 expert）。Mixtral 8x7B 有 47B 总参数，但每个 token 只用 13B。

```
稠密：每个 token → 全部权重
稀疏：每个 token → 部分权重（选中的 expert）
```

稀疏性的好处和代价

	好处	代价
训练	同样算力下能训练大得多的模型	负载均衡难、训练不稳定
推理	总能力更强	显存需求大、expert 可能闲置

> ⚡ **核心一句话：** 稠密 = 每次全用。稀疏 = 每次只用一部分。MoE 用稀疏性换来了更大的模型容量。

第 12 章　MoE 是什么

终于到了

有了前面 11 章的基础，MoE 就很好理解了：

```
MoE = Router + N 个 Expert（每个 Expert 就是一个 FFN）

Step 1: token 向量进入 Router
Step 2: Router 算出每个 Expert 的匹配分数
Step 3: 选 Top-K 个分数最高的 Expert
Step 4: 把 token 送给这 K 个 Expert 处理
Step 5: 加权合并 K 个 Expert 的输出
```

> 💡 **类比：** 你去医院看病。挂号台（Router）根据你的症状（token），判断你应该看内科还是骨科（选 expert）。复杂情况可能同时挂两个科（top-k=2）。每个医生都是某个领域的专家。

关键参数

参数	含义	常见取值
N	Expert 总数	8 (Mixtral) ~ 256 (DeepSeek-V3)
K	每个 token 激活的 expert 数	2 (Mixtral), 6-8 (DeepSeek)
稀疏度	1 – K/N	Mixtral: 75%, DeepSeek-V3: 97%

Router 怎么训练

Router 和 expert 一起训练。不需要告诉 router "这个 token 应该去哪个 expert"——router 自己通过反向传播学会分配。代价是容易"赢家通吃"：热门 expert 过载，冷门 expert 闲置。

负载均衡

为了防止 expert 旱涝不均，训练时加入 **auxiliary loss**（辅助损失）：如果某些 expert 用得太多或太少，就惩罚模型。DeepSeek 更进一步，用动态 bias 调整路由器偏好，不需要 auxiliary loss。

Shared Expert

DeepSeek 的创新：设一个"通用 expert"，所有 token 都经过它，外加 K 个"专业 expert"。这样通用知识不必在每个 expert 里重复存储。

> ⚡ **核心一句话：** MoE = 一群专家 + 一个路由器。每个 token 只找最对口的几个专家，既专业又省钱。

第 13 章　训练 vs 推理

对 MoE 来说是两回事

	训练	推理
瓶颈	计算量（FLOPs）	显存带宽
所有 expert 参与?	是（反向传播需要）	否（但都要加载到显存）
负载均衡	关键（影响训练效率）	不重要
主要成本	GPU 租金	显存 + 延迟

推理时 MoE 的显存问题

虽然推理时只计算 2 个 expert，但**所有 8 个（或 256 个）expert 都必须放在显存里**——因为你不知道下一个 token 会落到哪个 expert。这导致 MoE 模型显存需求远超同级别的稠密模型。

解法：

- **Expert Parallelism (EP)**：把不同 expert 分布到不同 GPU 上
- **Expert Offloading**：不常用的 expert 放 CPU 内存，用时加载

一句话总结

MoE 的 trade-off 很清晰：**用显存放能力，用稀疏换速度。**

第 14 章　层次化路由是什么

它是什么

当 Expert 数量很少（比如 8 个）时，Router 给每个 token 算一遍所有 expert 的分数，毫无压力。但如果 N = 2048 呢？每个 token 都要算 2048 次打分，Router 自己的计算量就爆炸了。

层次化路由就是把"一次选 2000 个"变成"层层筛选"。

> 💡 **类比：** 你要在一座 50 层的大楼里找一个人。一层一层、一间一间找，累死。但如果前台告诉你"他在 30 层"，30 层的楼层指引告诉你"他在 3002 室"——分两层筛选，工作量减少了几十倍。

怎么工作

```
第一层 Router：把 token 分给 16 个"专家组"
第二层 Router：在专家组内部，把 token 分给具体的 expert

   Token
    ↓
  Router 1（16 选 2）→ 选 2 个专家组
    ↓
  Router 2（每组 128 个 expert）→ 每组内选 2 个 expert
    ↓
  共 4 个 expert 处理，但路由计算量从 2048 次降到了 272 次
```

2017 年的 Sparsely-Gated MoE 论文就用这个方案支撑了上千个 expert。

为什么后来不用了

层次化路由的问题是复杂——两层 router 都要训练，都要做负载均衡。而且当 N = 256 时（DeepSeek-V3 的规模），直接 Top-K 的计算量完全可控，没必要搞这么复杂。层次化路由是"大 N 时代"的过渡方案，现在的硬件和算法让直接路由足够高效。

> ⚡ **核心一句话：** 层次化路由 = 给专家分组建索引，减少 Router 的计算量。N 不特别大时不需要。

第 15 章　专家容量是什么

它是什么

想象一个有 8 个医生的诊所。理论上每个医生每小时最多看 10 个病人。如果 Router（挂号台）把 50 个病人都挂给了医生 #3，医生 #3 看不过来。怎么办？

专家容量 = 每个 expert 最多能处理几个 token。

Capacity Factor

容量用一个叫 capacity factor 的参数控制：

```
每个 expert 的容量 = (总 token 数 / N 个 expert) × capacity factor

举个例子：
100 个 token, 8 个 expert, capacity factor = 1.2
每个 expert 最多处理 (100/8) × 1.2 = 15 个 token
```

Capacity Factor	含义
1.0	恰好均分，不允许任何专家多接
1.25	允许 25% 溢出缓冲，常用值
1.5	宽松，但显存浪费更多

容量溢出了怎么办

早期的粗暴做法（GShard、Switch Transformer）：**直接丢弃多余的 token**。这等于"挂不上号的病人直接赶出医院"。很浪费——token 是花了钱训练的。

后来的 improvement（ST-MoE 2022）：**随机路由给不太忙的 expert**。虽然不是最优匹配，但至少不丢弃。

> 💡 **类比：** 专家容量就是每个医生的"号源"。capacity factor = 1.0 意味着恰好平均分配（不现实），1.25 意味着允许热门医生多接 25% 的病人。

> ⚡ **核心一句话：** 专家容量是每个 expert 的"接诊上限"，用来防止少数 expert 过载。capacity factor 越大越安全，但越浪费显存。

第 16 章　细粒度专家切分是什么

它是什么

传统 MoE 的一个 expert 就是一个完整的 FFN。比如 Mixtral 8x7B，有 8 个 expert，每个 7B 参数。

DeepSeek 的洞见是：**把 1 个大 FFN 切成 m 个小 FFN（mini-expert）。**

```
传统 MoE (Mixtral):
  8 个 expert, 每个 = 1 个完整的 FFN (7B)
  每次激活 2 个 = 14B 参数

DeepSeek 细粒度切分 (m=4):
  1 个标准 FFN → 切成 4 个 mini-FFN (每个 ~1.75B)
  256 个 expert = 256 个 mini-FFN
  每次激活 8 个 mini-FFN ≈ 只用了 2 个"等效标准 FFN"的量
```

为什么要切分

更灵活的组合。原来 2 个大 expert 的组合方式有限（C(8,2) = 28 种）。256 个 mini-expert 选 8 个（C(256,8) ≈ 10^{14} 种），组合空间大了亿万倍。

> 💡 **类比：** 传统 MoE 像一个饭店只有 8 个大厨，每个都会做一类菜系。细粒度切分相当于把大厨拆成 256 个"配料师"——有人专切辣椒，有人专调酱汁。点一道菜时调用 8 个配料师组合，能搭配出更多口味。

代价

Router 要从 256 个 mini-expert 里选，计算量增加。但 DeepSeek 用了一系列工程优化（包括后面要讲的 Shared Expert），让这个开销可控。

> ⚡ **核心一句话：** 细粒度切分 = 把大专家切成小专家，让 Router 更灵活地组合。DeepSeek-V2/V3 的架构基石。

第 17 章　共享专家是什么

它是什么

前面第 12 章提了一嘴，这里展开讲。

细粒度切分之后，所有 expert 都是"专业 mini-expert"。但有一个问题：**语法、常识这类通用知识，每个 token 都需要，放哪个 expert 里?**

如果每个路由 expert 都存一份通用知识，浪费参数量。如果有某个 expert 专门存通用知识、但 Router 不一定每次都选它，那 token 可能"丢常识"。

DeepSeek 的方案：**分出一个或几个"共享专家"，所有 token 必须经过它们。**

```
Token 进入 MoE 层
  ├→ 共享专家（所有 token 必过，权重固定）→ 存语法、常识等"人人都需要"的知识
  └→ Router → 路由专家 #3, #7, #15... → 存代码、数学、特定语言等"特定 token 需要"的知识
                  ↓
          所有输出加权合并
```

> 💡 **类比：** 每个学生都要上"语文+数学"（共享专家 = 必修课）。然后根据不同兴趣，有人选修物理、有人选修历史（路由专家 = 选修课）。

好处

1. 通用知识不用在每个路由 expert 里重复存，省参数量
2. 共享专家的权重固定，不经过 Router 选择，保证每个 token 都拿到基础知识
3. DeepSeek-V2 的共享专家有 2 个，V3 有 1 个

> ⚡ **核心一句话：** 共享专家 = 必修课，路由专家 = 选修课。合用"必修+选修"消灭了知识冗余。

第 18 章　无辅助损失的负载均衡是什么

它是什么

回到第 12 章提到的负载均衡问题。

传统方案是加 auxiliary loss（辅助损失）：训练时多一个"惩罚项"——"专家 3 被用了 80% 的 token？罚！专家 7 只用 0.1%？罚！"

但 auxiliary loss 有个根本矛盾：它和主任务是对着干的。 主任务想让模型输出更准（降低 language loss），auxiliary loss 想让专家均匀分配。两个目标互相拉锯，就像你一边让车加速，一边踩刹车。

DeepSeek 的方案：动态 Bias

DeepSeek 放弃 auxiliary loss，改用了一种更优雅的方法：

```
每个 expert 的路由分数 = Router 原始分数 + bias

如果 expert #3 最近太忙 → bias_3 减一点 → token 不太愿意选 #3
如果 expert #7 最近太闲 → bias_7 加一点 → token 更愿意选 #7

bias 的调整只在"选 expert"这一步生效，不参与反向传播。
```

> 💡 **类比：** auxiliary loss 相当于"老板规定你必须平均分配精力给所有项目"——你可能在不擅长的项目上浪费时间。动态 bias 相当于"项目负责人根据每个人的工作量灵活调派"——你最近太忙就少给你派活，不耽误你把手头

> 的事干好。

为什么更好

bias 不进入梯度路径——只影响 token 去哪个 expert（前向），不影响模型学什么（反向）。模型的优化目标始终是"把语言模型做好"，负载均衡只是一个轻量的外界调节。

> ⚡ **核心一句话：** 动态 bias 用"温和的外力引导"替代了 auxiliary loss 的"强迫内耗"。DeepSeek-V2 之后的标准做法。

第 19 章　MLA（多头潜注意力）是什么

它是什么

这是一个比前面都硬核的概念，但它解决的问题很简单：**KV Cache 太大，显存放不下了。**

先回忆 Attention 的 KV Cache

Attention 层工作时，每个 token 要生成 K（Key）和 V（Value）向量存下来，供后续 token 查询。这些 K/V 向量占的显存叫 **KV Cache**。

长序列时，KV Cache 是推理显存的头号杀手。比如 128K 上下文，KV Cache 可能吃掉几十 GB 显存。

MLA 怎么压缩

MLA（Multi-Head Latent Attention）的核心理念是：**K 和 V 没必要存完整的，可以压缩到一个很小的"潜空间"（latent space）里，用的时候再解压。**

```
传统 Attention:
  Token → 生成完整的 K 向量 + V 向量 → 全量存进 KV Cache（占显存大）

MLA:
  Token → 压缩成一个很小的"潜向量"（latent vector）→ 只存潜向量（占显存小）
  需要用时 → 潜向量 × 解压矩阵 → 恢复出 K 和 V（精度损失很小）
```

> 💡 **类比：** 传统 Attention 相当于你把所有书都摊在桌面上（KV Cache 全量），找什么直接拿。MLA 相当于你只记每本书的索书号（潜向量），需要时去书架上按号取——桌面整洁多了。

压缩效果

DeepSeek-V2 的 MLA 将每个 token 的 KV Cache 从标准 MHA 的约 1.2MB 压缩到仅约 **70KB**——只有原来的 ~6%，对比主流模型用的 GQA 也降到了约 1/3~1/7（等效 GQA-2.25 组）。推理吞吐直接提升 5.76 倍。这个工程贡献

让 DeepSeek 的 MoE 模型在推理时省下的 KV Cache 显存，部分抵消了 MoE 需要加载所有 expert 的显存压力。

为什么和 MoE 放一起讲

MoE 让 FFN 参数膨胀（多 expert = 更多显存），MLA 让 Attention 显存收缩（压缩 KV Cache）。**DeepSeek 的完整方案是"MoE + MLA"的组合拳：FFN 端用稀疏换能力，Attention 端用压缩换显存。** 缺一个都玩不转。

> ⚡ **核心一句话：** MLA = 把 KV Cache 压缩存储、用时解压。和 MoE 配合使用，一胀一缩，总体显存可控。

第 20 章　KV Cache 是什么

它是什么

KV Cache 是 Transformer 推理时的一个"记忆缓存"。模型每生成一个新 token，都要回头"看"之前所有 token 的信息。如果不做缓存，每生成一个 token 就要把所有历史 token 重新算一遍——计算量按序列长度的平方增长，根本受不了。

KV Cache 的思路：把每个 token 算好的 **K（Key）和 V（Value）向量存下来**，生成下一个 token 时直接从缓存里读，不用重算。

> 💡 **类比：** 你写一篇文章，写到第 5000 个字时，不需要把前 4999 个字重新读一遍——你已经记住了前面写了什么。KV Cache 就是模型的"短期记忆本"。

为什么是显存杀手

KV Cache 的大小 = `2 × 层数 × KV head 数 × head 维度 × 序列长度 × 精度字节数`。

一个 671B 参数模型，在 128K 上下文下，KV Cache 可能吃掉 **几十 GB 显存**——比模型权重本身还大。

为什么 MoE 特别在意它

MoE 让 FFN 参数膨胀（多了 N 个 expert），但 **Attention 层的 KV Cache 开销不变**。也就是说，MoE 虽然省了 FFN 计算，但 KV Cache 的显存压力一点没减。这是 MoE 推理时显存压力的主要来源之一。

所有"压缩 KV Cache"的技术——MLA、DSA、HCA、GQA——本质上都是在解决同一个问题：怎么让这个"记忆本"变薄，同时不丢重要信息。

> ⚡ **核心一句话：** KV Cache = 模型生成时的短期记忆。KV Cache 越大推理越慢、越吃显存。压缩 KV Cache 是 MoE 推理优化的核心战场。

第 21 章　RLHF / PPO / GRPO 是什么

它们的关系

这三个概念是"套娃"关系：

```
RLHF（人类反馈强化学习）→ 是一个训练阶段
  └ PPO（近端策略优化）→ 是 RLHF 里最常用的训练算法
      └ GRPO（群体相对策略优化）→ 是 PPO 的改进版，DeepSeek R1 提出
```

RLHF 是什么

模型预训练完之后，虽然"懂很多"，但不太会"好好说话"。RLHF 的目标是用人类的偏好来引导模型：同样的问题，人类更喜欢哪个回答？让模型学着往人类偏好的方向调整。

> 💡 **类比：** 预训练 = 小孩读了整个图书馆（什么都知道但说话没分寸）。RLHF = 家长在旁边不断纠正"这样说不对，这样说更好"。

PPO 是什么

PPO 是 RLHF 阶段最常用的训练算法。它的核心设计是：**每次更新步子不要太大**——防止模型为了"讨好"奖励模型而忘掉预训练学到的知识。

PPO 需要同时跑四个模型：策略模型（正在训练的）、参考模型（训练前的快照）、奖励模型（打分员）、Critic 模型（价值评估）。四个模型都要占显存，训练成本巨大。

GRPO 是什么

DeepSeek R1 提出的改进。GRPO 的核心洞察：**Critic 模型可以去掉**。

怎么做？对同一个问题，让模型生成一组候选答案（比如 4 个），以组内平均分为基准算相对优势。得分比组内平均高的就是"好答案"，低的就"需要改进"。不需要一个独立的 Critic 模型来评估每个回答的"价值"。

> 💡 **类比：** PPO = 请一个专职评委（Critic 模型）给每个学生打分。GRPO = 让学生们互相对比——不需要专职评委，只要知道谁比组内平均分高就行。

省掉 Critic 模型意味着：训练显存减半、训练速度加倍。

GRPO 是 DeepSeek R1 能做出"R1-Zero"（纯 RL、零 SFT 数据）的关键工程前提。 它让 RL 训练的成本降到可以做大规模实验的程度。

> ⚡ **核心一句话：** RLHF = 用人类偏好引导模型。PPO = RLHF 的经典算法，需要 Critic 模型。GRPO = PPO 的改进版，去掉 Critic 模型用组内对比替代，DeepSeek R1 的核心工程创新。

第 22 章　线性注意力是什么

它是什么

标准 Softmax Attention 的计算复杂度是 **$O(L^2)$**——序列长度翻倍，计算量翻四倍。长文本时这成了瓶颈。

线性注意力的目标是让复杂度降到 **$O(L)$**——序列长度翻倍，计算量只翻一倍。

怎么做

核心思路是**改变计算顺序**。标准 Attention 必须先算所有 token 对之间的"相关性分数"（L × L 矩阵），再乘以 V。线性注意力把这个过程反过来——用"隐状态"先累积历史信息，来一个新 token 直接更新隐状态。

有哪些方案

方案	提出者	做法
MLA	DeepSeek (2024)	把 KV 压缩到"潜空间"，间接减少 KV Cache
DSA	DeepSeek (2025)	用 Indexer 打分只选 top-k 个最相关历史 token
GatedDeltaNet	Qwen3-Next (2025)	用 delta 规则更新隐状态，选择性记住/忘记
HCA	DeepSeek V4 (2026)	激进压缩：每 128 token 合成一个
SWA+GA	MiMo (2026)	滑动窗口 + 全局注意力 6:1 交替

> 💡 **类比：** 标准 Attention = 你要跟 1000 个人分别打招呼（1000×1000 次互动）。线性注意力 = 你维护一个"当前总结"，来一个新面孔更新一下总结就行（1000 次更新）。

为什么和 MoE 放一起讲

MoE 解决了 FFN 层的稀疏化，线性注意力解决了 Attention 层的稀疏化。两者合在一起，才能做出真正"又大又省"的模型。

DeepSeek V4 的 Hybrid Attention（CSA+HCA）、Qwen3-Next 的 GatedDeltaNet、MiMo 的 SWA+GA——2025-2026 年几乎所有 MoE 模型的架构创新都围绕"注意力怎么省"展开。因为这已经是 MoE 推理最后一道没有攻破的性能墙。

> ⚡ **核心一句话：** 线性注意力 = 让模型看长文时计算量从 $O(L^2)$ 降到 $O(L)$。和 MoE 配合，一省 FFN、二省 Attention，合在一起才是完整的"省电方案"。

> 理解了以上 22 个概念，你就有了读懂任何 MoE 论文（包括 DeepSeek-V4 技术报告）的基础。

> 下一步建议：先读 Mixtral 8x7B 论文（ArXiv: 2401.04088）入门，再读 DeepSeek-V2/V3 技术报告感受架构创新，最后读 DeepSeek-V4 技术报告了解最新进展。